

Chas Academy
Fullstack .NET
Thesis 2026

Glint: A Multi-Method Web Application for Real-Time Resume and Job Advertisement Analysis

Fredrik Andersson & Max Berglund
Supervisor: Max Guclu

CHAS ACADEMY AB
Arenavägen 61
121 77 Johanneshov, Sweden



ABSTRACT

This thesis presents the design and implementation of Glint, a full-stack web application that enables job seekers to evaluate the alignment between their CV and a specific job advertisement. The system accepts a user-uploaded PDF resume and a pasted job description, and analyses the match using three independent, concurrently executing methods: a semantic AI analysis powered by Google Gemini, a rule-based structural evaluation, and a domain-agnostic keyword-frequency analysis. The frontend was built in Next.js with TypeScript and Tailwind CSS; the backend in ASP.NET Core (.NET 8) with Entity Framework Core targeting a PostgreSQL database. Authentication uses short-lived JWTs with rolling refresh tokens, supplemented by one-time codes delivered via email. Results are streamed to the client in real time via Server-Sent Events, with each method's score rendered as it becomes available. The project demonstrates that a multi-method approach produces a more robust and nuanced match score than any single strategy in isolation, and identifies file storage architecture and automated test coverage as the primary areas for future production hardening.

LIST OF ABBREVIATIONS AND CONCEPTS

Abbreviation	Definition
AI	Artificial Intelligence - machine-based systems capable of performing tasks that typically require human reasoning, such as language understanding and pattern recognition.
API	Application Programming Interface - a defined contract that allows software components to communicate with each other.
ASP.NET Core	A cross-platform, open-source web framework developed by Microsoft for building web applications and APIs on the .NET runtime.
ATS	Applicant Tracking System - software used by employers to filter and manage job applications automatically, often based on keyword matching.
CI/CD	Continuous Integration / Continuous Deployment - a development practice in which code changes are automatically built, tested, and deployed on each commit.
CV	Curriculum Vitae - a document summarising a person's professional experience, education, and skills.
DTO	Data Transfer Object - a plain object used to carry data between layers of an application, typically between the API and the client.
EF	Entity Framework - Microsoft's object-relational mapping (ORM) framework for .NET, used to interact with databases using C# objects instead of raw SQL.
JWT	JSON Web Token - a compact, URL-safe token format used to transmit claims between parties, commonly used for stateless authentication.
LLM	Large Language Model - a type of AI model trained on large volumes of text, capable of generating and interpreting natural language.

MIME	Multipurpose Internet Mail Extensions - a standard that indicates the nature and format of a file, used in HTTP to identify content types such as application/pdf.
MVP	Minimum Viable Product - the earliest version of a product that includes only the features necessary to demonstrate core functionality.
Next.js	A React-based frontend framework that supports server-side rendering, static generation, and the App Router file-system routing model.
ORM	Object-Relational Mapper - software that translates between object-oriented code and relational database tables.
OTC	One-Time Code - a short-lived numeric code sent to a user's email address, used for verification or passwordless login.
PDF	Portable Document Format - a file format developed by Adobe that presents documents consistently across devices and operating systems.
React	A JavaScript library for building user interfaces, developed by Meta. Next.js is built on top of React.
SSE	Server-Sent Events - a web standard that allows a server to push data to a browser over a single HTTP connection, used here to stream analysis results in real time.
TypeScript	A statically typed superset of JavaScript that compiles to plain JavaScript, providing compile-time type checking.
XSS	Cross-Site Scripting - a class of web security vulnerability in which an attacker injects malicious scripts into a web page viewed by other users.
Bytea	A PostgreSQL binary data type used to store raw byte sequences, such as PDF file contents, directly within a database column

TABLE OF CONTENTS

ABSTRACT.....	2
LIST OF ABBREVIATIONS AND CONCEPTS.....	3
TABLE OF CONTENTS.....	5
1. INTRODUCTION.....	6
1.1 BACKGROUND.....	6
1.2 PURPOSE.....	7
1.3 DELIMITATIONS.....	7
2. METHOD AND IMPLEMENTATION.....	8
2.1 Project Planning and Architecture Design.....	8
2.2 Division of Responsibilities.....	11
2.3 Backend Initialisation and Domain Modelling.....	11
2.4 Frontend Initialisation and MVP Scaffolding.....	12
2.5 Authentication and Email Verification.....	13
2.6 File Validation and Resume Management.....	13
2.7 Analysis Pipeline and Real-Time Streaming.....	14
2.7.1 Concurrent Execution.....	14
2.7.2 PDF Text Extraction.....	15
2.7.3 AI Semantic Analysis.....	15
2.7.4 Rule-Based Analysis.....	16
2.7.5 Keyword-Frequency Analysis.....	16
2.8 User Dashboard and Data Management.....	17
2.9 User Experience and Frontend Refinements.....	18
2.10 Deployment.....	18
2.11 Challenges.....	19
3. RESULTS.....	20
3.1 Authentication and User Management.....	20
3.2 CV and Job Advertisement Management.....	21
3.3 PDF Text Extraction.....	21
3.4 User Dashboard.....	21
3.5 Guest Access.....	22
3.6 Analysis Pipeline.....	22
4. DISCUSSION.....	24
4.1 Analysis Methods.....	24
4.2 Known Limitations.....	25
4.3 Architectural Decisions.....	26
4.4 Deployment.....	26
4.5 Future Prospects.....	27
5. CONCLUSION.....	27
6. SOURCE LIST.....	29
APPENDICES.....	30
Appendix A:.....	30
Appendix B:.....	30

1. INTRODUCTION

1.1 BACKGROUND

The modern labour market is increasingly characterised by the widespread adoption of Applicant Tracking Systems (ATS), automated recruitment platforms that filter incoming applications based on keywords and structural criteria long before a human recruiter reviews them. A landmark study by Fuller et al. (2021) found that ATS software systematically screens out large numbers of qualified candidates because their CVs do not match the precise terminology or formatting the system expects, even when those candidates possess the underlying skills a role demands. The study further found that this problem is widely acknowledged by employers themselves, with the majority of executives surveyed recognising that their own screening tools were causing them to overlook viable applicants (Fuller et al., 2021).

A number of CV analysis tools exist commercially, among them Jobscan and Rezi. Jobscan operates primarily by comparing keyword frequency between a CV and a job posting, generating a match rate percentage based on the presence or absence of specific terms drawn directly from the job description (Jobscan, 2025). Rezi similarly evaluates resumes against a fixed set of 23 ATS-critical checkpoints, covering formatting rules and keyword density (Rezi, 2026). While both tools help candidates identify missing keywords, neither accounts for semantic meaning or contextual relevance, and the approach risks incentivising keyword stuffing over a demonstration of genuine competence. Accessible platforms that combine multiple complementary analytical methods to produce a nuanced and transparent result remain scarce.

Glint addresses this gap by integrating three distinct analysis strategies within a single system utilizing semantic AI analysis, rule-based evaluation, and frequency-driven keyword extraction, presenting their results to the user simultaneously in real time. The project constitutes a full-stack application and was carried out by Fredrik Andersson and Max Berglund as part of the Fullstack .NET programme at Chas Academy.

1.2 PURPOSE

The purpose of this thesis is to design and implement a full-stack web application that analyses and evaluates, in real time, the degree of alignment between a user-uploaded resume and a provided job advertisement. This is achieved through three concurrently executing analysis methods: AI semantic analysis, rule-based evaluation, and keyword-frequency matching, including a critical reflection on the strengths and limitations of each approach both individually and in combination.

1.3 DELIMITATIONS

All algorithms and the underlying text processing pipeline are designed exclusively for English-language content. Both the CV and the job advertisement must therefore be written in English for the analysis results to be reliable; support for additional languages falls outside the scope of this project.

Furthermore, each user account is restricted to saving a maximum of five CV files and five job advertisements. This constraint is grounded in practical database storage considerations, as PDF files are currently persisted directly within the relational database as binary data (bytea). The implications of this architectural decision are examined in greater detail in the Discussion section.

The analysis methods implemented in this project do not claim to constitute an exact or exhaustive science. The keyword-based analysis operates on a curated vocabulary of English terms and is inherently bounded by the completeness of that vocabulary. Similarly, the AI-based analysis is dependent on the quality of the prompt supplied to the language model, and the time available for iterative prompt engineering was limited by the overall project timeline. Both methods should therefore be understood as approximations rather than definitive evaluations. The application's analysis engines are intended as decision-support tools and qualitative aids; they do not constitute a guarantee of any employment outcome and are not a substitute for professional career guidance.

Finally, several ancillary features identified during development fall outside the scope of the submitted application. These include a functional contact form submission endpoint, search engine meta optimisation, and a more robust authentication token storage strategy. These are discussed further in the Discussion section. Access tokens are currently stored in localStorage, which exposes them to cross-site scripting (XSS) attacks (OWASP Foundation, 2021) should malicious script execute in the browser context; a more secure alternative using httpOnly cookies falls outside the scope of this project.

2. METHOD AND IMPLEMENTATION

2.1 Project Planning and Architecture Design

The project commenced with a collaborative planning phase in which both team members designed the relational database schema prior to writing any code. This decision was deliberate: by agreeing on the data model early, the team could work against a shared and stable reference point throughout development. A database diagram tool (dbdiagram.io) was used to produce a visual schema, which served as the primary reference going forward.

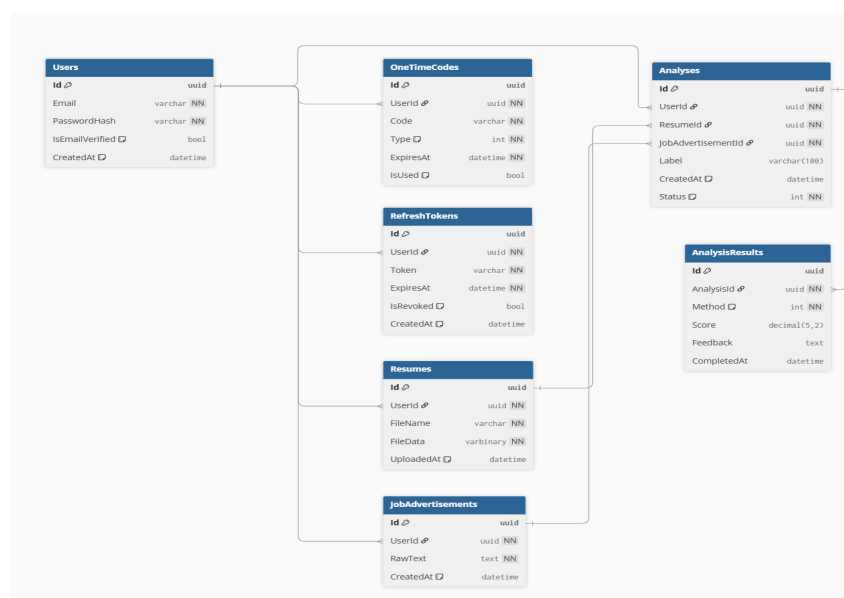


Figure 1: The relational database schema designed during the planning phase, showing all entities (Users, Resumes, JobAdvertisements, Analyses, AnalysisResults, OneTimeCodes, and RefreshTokens) and their relationships. Screenshot taken by the authors, May 2026.

To organise the project, a shared GitHub Organisation was created, grouping two separate repositories: one for the frontend and one for the backend. This separation reflects the architectural boundary between the client-side and server-side concerns and allowed both developers to work in parallel with minimal merge conflicts.

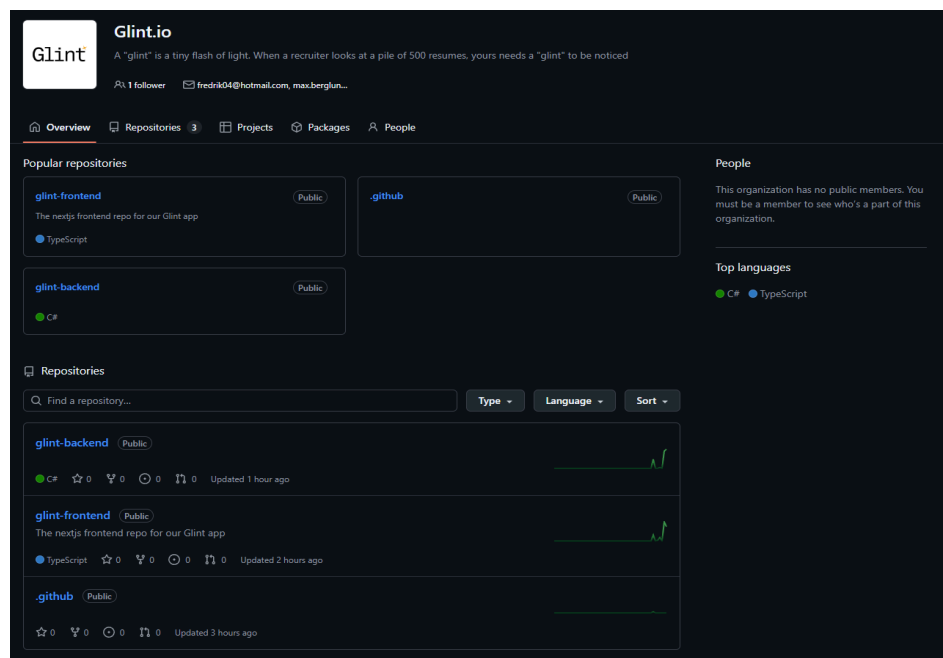


Figure 2: The Glint GitHub Organisation page, containing the two separate repositories for the frontend (Next.js/TypeScript) and backend (C#/.NET). Screenshot taken by the authors, May 2026.

The application was divided into two primary subsystems. The frontend was built using the Next.js framework with TypeScript and Tailwind CSS. The backend was implemented using ASP.NET Core (.NET 8), with Entity Framework Core (EF Core) handling data persistence against a relational database. During local development this targeted Microsoft SQL Server; prior to deployment the provider was migrated to PostgreSQL to match the managed database offering on the chosen hosting platform, as described in Section 2.10. A code-first approach was adopted for the database, meaning the database schema was derived from C# model classes annotated with data annotations, rather than being designed manually in SQL and then mapped to code.

This approach kept the schema and the application code in synchronisation throughout development and simplified the migration workflow. A full inventory of third-party libraries and NuGet packages used in the project is provided in [Appendix A](#).

2.2 Division of Responsibilities

The work was divided so that one team member took primary responsibility for the backend and the other for the frontend. This division allowed both areas to progress in parallel and reduced the risk of the two developers working on the same files at the same time. In practice the boundary was not rigid, and both developers contributed across both sides as the project progressed, but the initial setup and the majority of the core implementation in each area followed this split. The subsections below reflect this structure, though the work should be understood as a joint effort throughout.

2.3 Backend Initialisation and Domain Modelling

The backend was initialised by one team member. The domain models (User, Resume, JobAdvertisement, Analysis, AnalysisResult, OneTimeCode, and RefreshToken) were defined as C# classes in the Models folder. Each model was annotated with data annotations such as [Required], [Key], [ForeignKey], and [MaxLength] to define validation rules and relational constraints directly in code.

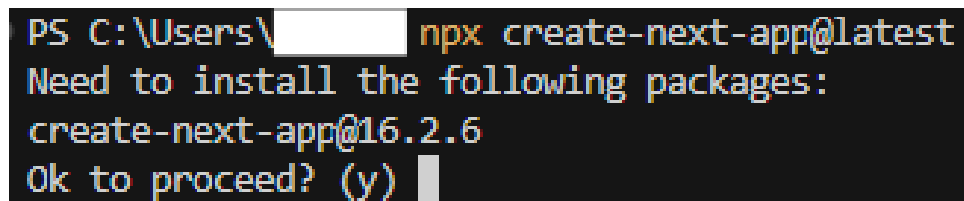
Entity Framework Core then interpreted these annotations to generate the corresponding database schema via migrations. A notable design decision was the use of an AnalysisStatus enum (Pending, InProgress, Completed) on the

Analysis entity, and an AnalysisMethod enum (AI = 0, RuleBased = 1, Keyword = 2) on the AnalysisResult entity. Enums are stored as their underlying integer values in the database, which is safer than persisting raw strings that are prone to casing inconsistencies and typographical errors. The AppDbContext class was configured to establish the connection between the application and the database, and cascade delete behaviour was explicitly set to NoAction on the Analysis-Resume and Analysis-JobAdvertisement relationships to prevent unintended data loss when a resume or job advertisement is removed.

The backend folder structure followed a Repository-Service pattern, organised into Controllers, Services, Repositories, Interfaces, DTOs, Models, and Exceptions. This separation of concerns ensures that database access logic is isolated in repositories, business logic resides in services, and controllers remain thin orchestrators that delegate to the service layer. This architecture facilitates testing and makes it straightforward to swap implementations, for instance replacing a concrete repository with a mock during unit testing. To support this from the outset, a DbSeeder class was introduced that runs on application startup and populates the database with representative users, resumes, job advertisements, analyses, and analysis results if no data is present. This ensured that development and debugging could proceed against realistic data from the earliest stages of the project, without requiring manual database population each time the application was restarted.

2.4 Frontend Initialisation and MVP Scaffolding

The frontend was initialised by the other team member. The Next.js project was created using the official scaffolding



```
PS C:\Users\[redacted] npx create-next-app@latest
Need to install the following packages:
create-next-app@16.2.6
Ok to proceed? (y) [redacted]
```

Figure 3: Terminal output of the `npx create-next-app@latest` command used to scaffold the Next.js frontend project. Screenshot taken by the authors, May 2026.

This produced a project with TypeScript support, Tailwind CSS, and the App Router structure. The initial pages: home, login, user dashboard, analysis, about, and contact: were created based on the planned feature set and wired together using Next.js's file-system routing.

To unblock frontend development before the backend was fully operational, a local mock data layer was introduced in `app/lib/mock-data.ts`. This file defines representative TypeScript types mirroring the backend models and exports hardcoded instances of users, resumes, job advertisements, analyses, and analysis results.

The `data-source.ts` module wraps all data-fetching logic with a fallback strategy: if the `NEXT_PUBLIC_API_BASE_URL` environment variable is not set, all data operations fall back to the mock layer. This allowed the frontend to be developed and styled independently, with real API integration introduced progressively as backend endpoints became available.

2.5 Authentication and Email Verification

The authentication system supports user registration, email verification, login, one-time code login, token refresh, and password reset. The flow proceeds as follows.

Upon registration, the user's password is hashed using BCrypt before being stored. A six-digit one-time code is generated and persisted in the `OneTimeCodes` table with a fifteen-minute expiry and a type of `EmailVerification`. The code is delivered to the user's registered email address via an HTML email constructed by a static builder method in `EmailService`. An initial implementation that used MailKit over SMTP was discarded after it produced styling inconsistencies in the rendered emails; the final implementation uses the Resend HTTP email delivery service, which resolved these issues and simplified the integration. The email presents two redemption paths: a clickable verification button and a manual code entry field, supporting both desktop and mobile clients.

Session management is handled through short-lived JSON Web Tokens (Jones, Bradley and Sakimura, 2015) with a fifteen-minute expiry, combined with rolling refresh tokens valid for seven days. The JWT carries the user's identifier and email address as claims. Upon each token refresh, the existing refresh token is explicitly revoked server-side before a new one is issued, preventing token reuse following session renewal. OTC-based login follows the same issuance pattern, enabling passwordless authentication via a code sent to the user's inbox.

2.6 File Validation and Resume Management

PDF uploads are processed through a dedicated `FileValidationService` before any data is persisted. The validation pipeline applies the following checks in sequence: a null and empty guard, a file size check enforcing a maximum of five megabytes, a MIME type check requiring `application/pdf`, a magic byte inspection verifying that the file begins with the PDF signature bytes `%PDF (0x25 0x50 0x44 0x46)`, and a content scan that decodes the file as Latin-1 and searches for known malicious PDF patterns, including embedded JavaScript keywords (`/JavaScript`, `/JS`), launch actions (`/Launch`, `/OpenAction`), and obfuscation primitives (`eval`, `unescape`). Latin-1 encoding is used for the scan rather than UTF-8 because it maps every byte to exactly one character, ensuring no content is silently dropped during decoding.

Each authenticated user is permitted a maximum of five saved resumes. If a user attempts to upload a sixth file, an appropriate error response is returned prompting them to delete an existing file first. Resume binary data is stored directly in the database as a `bytea` column. The trade-offs of this approach are discussed further in Section 4.

2.7 Analysis Pipeline and Real-Time Streaming

The analysis pipeline is the core component of the application. It accepts a PDF document and a job advertisement text as input and produces three independent scores and feedback payloads, one per analysis method.

2.7.1 Concurrent Execution

All three methods: AI semantic analysis, rule-based evaluation, and keyword-frequency analysis: are dispatched concurrently using `Task.WhenAll`. This means the total wall-clock time for an analysis request is bounded by the slowest individual method rather than the sum of all three, which is particularly important given that the AI method involves a network call to an external language model API. Results are streamed to the frontend in real time using Server-Sent Events (WHATWG, 2026) delivered over an unbounded in-memory channel.

Each method writes its result to the channel as soon as it completes, and the SSE endpoint reads from that channel and flushes events to the connected client. This architecture ensures that the user sees each score appear in the interface as it becomes available, rather than waiting for all three methods to complete before receiving any feedback.

2.7.2 PDF Text Extraction

Before the analysis methods can operate, the text content of the uploaded PDF must be extracted. This is performed server-side using the PdfPig library. The service iterates over each page of the document and concatenates all extracted word-level text tokens into a single string, which is then passed to each analysis method. If the document contains no extractable text: for instance, if it consists entirely of scanned images: the extraction returns an empty string rather than raising an unhandled exception, and downstream methods receive an explicit signal indicating the absence of usable content.

An initial attempt to write a custom PDF parser was abandoned after it failed to correctly handle UTF-8 encoded characters, producing replacement characters for non-ASCII content. PdfPig resolved this issue reliably.

2.7.3 AI Semantic Analysis

The AI analysis method submits the extracted CV text and the job advertisement text to the Google Gemini language model via a structured prompt. The model selected was **gemini-3.1-flash-lite-preview**, chosen after evaluating several alternatives. Earlier candidates including **gemini-2.5-pro** were found to be unsuitable due to token input limits and request-rate constraints that could not accommodate the volume of text submitted in a typical analysis request without paid access. As no external funding was available for the project, the model selection was constrained to free-tier offerings.

Prompt engineering was a significant focus of this phase. The final prompt instructs the model to return a single Match Score: [n] line at the very top of its response, followed by two fixed sections: Missing Elements and Actionable Fixes, each in Markdown format. Preambles and closing remarks were explicitly prohibited to reduce output variability. Earlier iterations that allowed free-form output produced inconsistent score extraction, as the numeric score appeared in different positions across responses; pinning it to the first line resolved this. A regular expression was then applied on the backend to extract the score before the remainder was forwarded to the client as the feedback payload.

2.7.4 Rule-Based Analysis

The rule-based method uses the same Gemini model to perform a deterministic, structured evaluation of the CV against the job advertisement. The prompt instructs the model to first extract the requirements from the job advertisement, classify each as either a must-have (weight 2) or preferred (weight 1) criterion, and then verify which of those criteria are evidenced in the CV. The model is required to return only a valid JSON object conforming to a defined schema, which the backend deserialises into a typed RuleEvaluationResponse model. This approach combines the flexibility of natural language understanding for requirements extraction with the predictability of structured output for score calculation.

Both AI-driven services implement a single-retry mechanism for 429 Too Many Requests responses, pausing for a fixed three-second delay before the retry attempt. If the second attempt also fails, or if the error indicates a hard quota limit, an AiServiceUnavailableException is raised immediately rather than retrying further.

2.7.5 Keyword-Frequency Analysis

The keyword-frequency method operates without the language model and is therefore fully deterministic and not subject to external API availability. Tokens are extracted from the requirements section of the job advertisement and classified as either domain-specific or generic using a curated vocabulary list of common English words. Domain-specific terms are assigned twice the weight of generic terms in the final score calculation.

Bigrams: two-word sequences: are additionally extracted and matched against the CV at a blended weighting, improving recall for compound technical terms such as "machine learning" or "software architecture".

The method returns a structured payload containing matched terms, missing terms, and a summary string. In the user interface, matched and missing terms are rendered as colour-coded tag elements, giving the user immediate visual feedback on which job advertisement keywords are and are not represented in their CV.

The final score combines these signals as $\text{score} = (\text{tokenScore} \times 0.90) + (\text{bigramScore} \times 0.10)$, clamped to $[0, 100]$, where tokenScore is the ratio of matched weighted token points to total weighted token points, with important tokens counted at 2× and general tokens at 1×.

The accuracy of this method required significant iteration. Initial testing against a real job advertisement yielded a score of 15 out of 100 for a CV that was a plausible match for the role, indicating that the filtering logic was either too aggressive or failing to normalise terms correctly. A key identified cause was the failure to recognise semantically equivalent but syntactically distinct representations of the same term, for example "C-sharp" versus "C#". The filtering and normalisation logic was revised to address the most common such cases, and subsequent testing demonstrated a measurable score increase when the same CV was rewritten using the specific terminology present in the job advertisement, validating that the method correctly rewards keyword alignment.

2.8 User Dashboard and Data Management

The user dashboard provides authenticated users with an overview of their analysis history, aggregate statistics, and a score-over-time chart. The statistics endpoint aggregates scores from the AnalysisResults table grouped by method, computing average scores and counts per method as well as a chronological series of data points for the trend chart, which is rendered client-side using Chart.js.

History records can be filtered by time range and deleted individually or in bulk. Deletion granularity options include the current day, the past seven days, the past thirty days, the past year, and all time. A complete account deletion endpoint was

also implemented, removing the user's profile and all associated relational data. Job advertisements submitted during analysis are automatically persisted per user, subject to a maximum of five stored entries. When this limit is exceeded, the oldest entry is replaced and a notification is surfaced to the user.

2.9 User Experience and Frontend Refinements

Following the completion of the core functional pipeline, several rounds of frontend refinement were undertaken. Navigation links are rendered conditionally based on authentication state, directing unauthenticated users toward the analysis and registration flows. A toast notification system was introduced using the React-Toastify library to provide non-blocking feedback for actions such as file uploads, analysis completion, and deletion confirmation. The analysis page was made fully responsive for mobile viewports, and consistent visual design was enforced across all pages, including the landing page, dashboard, and authentication screens. A privacy policy and terms of service were added ahead of deployment to satisfy basic data governance requirements for a user-facing application.

2.10 Deployment

The frontend was deployed to Vercel via continuous integration connected to the frontend GitHub repository, enabling automatic preview and production deployments on each push to the main branch.

The backend was deployed to Railway. During development the application targeted a local Microsoft SQL Server instance, but Railway's managed database offering uses PostgreSQL, requiring a database provider switch from `Microsoft.EntityFrameworkCore.SqlServer` to `Npgsql.EntityFrameworkCore.PostgreSQL` ahead of deployment. The migration was straightforward as Entity Framework Core abstracts most provider-specific behaviour, though column type mappings for Guid primary keys and decimal precision fields required minor annotation adjustments.

The application reads its target port from the PORT environment variable at startup, which Railway injects automatically. Database migrations are applied on startup via MigrateAsync(), ensuring the schema is always current without manual intervention between deployments. All sensitive configuration - database connection strings, JWT parameters, Gemini API keys, and Resend credentials - is managed through Railway's environment variable interface and is never committed to source control.

2.11 Challenges

Several significant challenges were encountered during the project. The most substantial difficulties arose in connection with the analysis methods. Selecting a suitable AI model proved particularly challenging because no budget was available for paid API access, and the models available under free tiers varied considerably in their token limits, request quotas, and output reliability. Several candidates were evaluated and discarded before a model capable of processing the combined length of a CV and a job advertisement within a single prompt was identified.

The keyword analysis method presented a separate class of challenges rooted in natural language variability. Technical terms in job advertisements are frequently expressed in multiple forms: abbreviations, symbols, and expanded names may all refer to the same concept: and the initial algorithm did not account for this, producing scores that penalised candidates whose CVs used equivalent but differently formatted terminology. Addressing this required careful curation of the normalisation rules and iterative testing against real job advertisements and CV documents to validate improvements.

A further challenge involved the real-time streaming architecture. An early version of the frontend polling logic failed to reflect results from the faster analysis methods until the slowest method had also completed, negating the intended progressive disclosure experience. This was resolved by restructuring the SSE channel consumption logic to flush each result to the client immediately upon receipt, independent of the other methods' completion status.

3. RESULTS

The result of the project is a functioning full-stack web application named Glint, consisting of a Next.js frontend and an ASP.NET Core (.NET 8) backend connected to a PostgreSQL database. The application is accessible via a web browser and supports both guest and authenticated usage flows.

3.1 Authentication and User Management

The application implements a complete authentication system supporting user registration, email verification via one-time codes (OTC), login, password reset, and account deletion. The email verification step requires the user to enter a six-digit code delivered to their registered address, as illustrated in Figure 4. Session persistence is achieved through short-lived JSON Web Tokens (Jones, Bradley and Sakimura, 2015) with an expiry of 15 minutes, automatically renewed via rolling refresh tokens with a validity period of seven days. Upon each token renewal, the previous refresh token is explicitly revoked server-side via a dedicated repository call, after which a new refresh token is issued and stored. This is designed to prevent a compromised token from being reused after a session has been renewed.

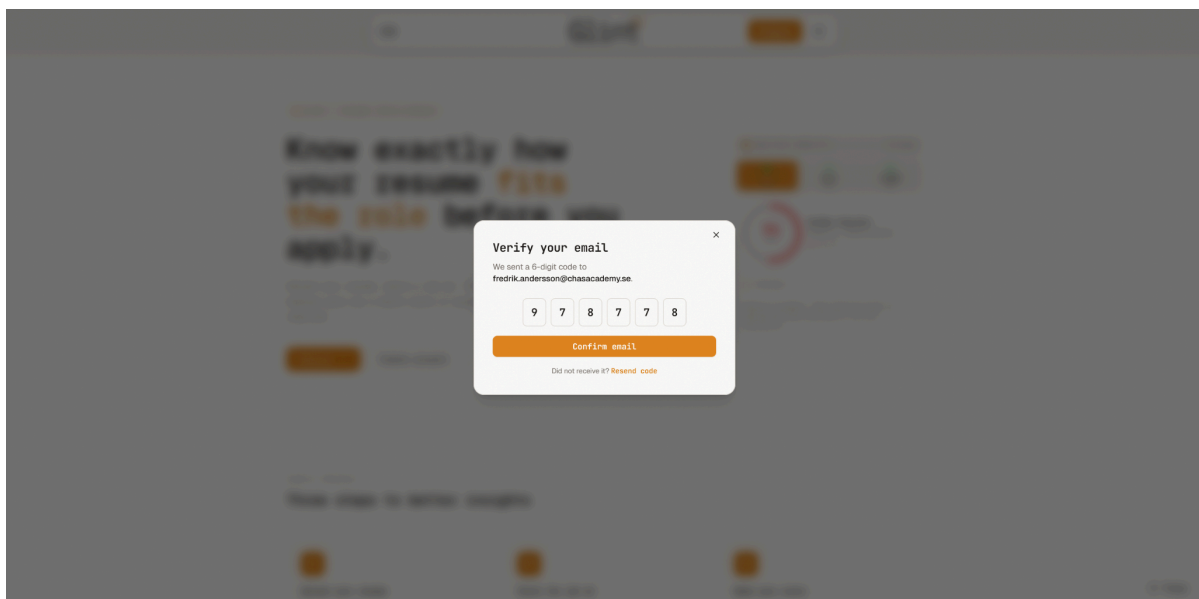


Figure 4: The email verification screen displaying the six-digit one-time code (OTC) input interface. Screenshot taken by the authors, May 2026.

3.2 CV and Job Advertisement Management

Authenticated users may upload PDF resumes of up to 5 MB in size, with a maximum of five files stored per account. Uploaded files are validated against a set of security criteria prior to storage, including MIME type verification, magic byte inspection, and a scan for known malicious PDF content patterns such as embedded JavaScript. Job advertisements submitted during analysis are automatically persisted per user account, also subject to a maximum of five entries, with the oldest entry replaced when the limit is reached.

3.3 PDF Text Extraction

PDF text extraction is performed server-side using the PdfPig library (version 0.1.14). The service iterates over each page of the uploaded document and concatenates the extracted text into a single string, which is then passed to the analysis pipeline. In cases where the PDF is invalid or contains no extractable text, the service returns an empty result rather than raising an unhandled exception.

3.4 User Dashboard

Authenticated users have access to a dashboard presenting an overview of their analysis history, aggregate statistics including total analyses performed and average score per method, and a score-over-time chart rendered using Chart.js, as shown in Figure 5. Historical analyses can be filtered by time range and cleared individually or in bulk. Saved resumes and job advertisements can be previewed and deleted from the same interface.

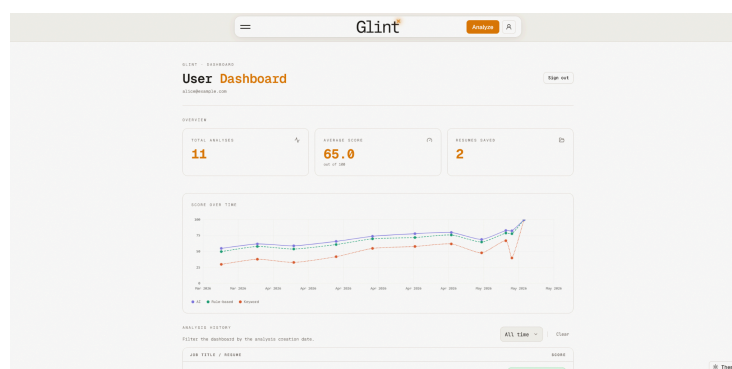


Figure 5: The authenticated user dashboard displaying the score-over-time chart and analysis history table. Screenshot taken by the authors, May 2026.

3.5 Guest Access

Users who have not registered an account may perform analyses without any restriction on quantity. In guest mode, a PDF file must be uploaded directly with each request, as no saved resumes or job advertisements are available to reference. No data from the analysis is persisted server-side; the pipeline executes transiently and returns results directly to the client. All three analysis methods are available to guest users on equal terms with authenticated users.

3.6 Analysis Pipeline

The core analysis pipeline accepts a PDF document and a job advertisement as input and executes three independent analysis methods concurrently using `Task.WhenAll`. Results are streamed to the frontend in real time via Server-Sent Events (SSE) using an unbounded in-memory channel, such that each method's score and feedback are rendered in the user interface as soon as that method completes, regardless of the completion order of the other methods. Figure 6 illustrates the interface mid-analysis, with one method having completed while the remaining two are still processing.

The three methods produce the following outputs:

AI Semantic Analysis: The extracted CV text and job advertisement text are submitted to the Google Gemini language model via a structured prompt. The model returns a match score between 0 and 100 and a set of actionable recommendations, including identified skill gaps and specific rewrite suggestions.

Rule-Based Analysis: The CV text is evaluated against a set of structural and content criteria derived from the job advertisement, producing a weighted score and structured feedback identifying passed and failed criteria, categorised by severity.

Keyword-Frequency Analysis: Tokens are extracted from the job advertisement's requirements section and classified as either domain-specific or generic based on a curated common English vocabulary list. The CV is then evaluated for coverage of these tokens, with domain-specific terms weighted at twice the value of generic terms. Bigrams are additionally extracted and matched at a blended weighting. The method returns a structured payload containing matched terms, missing terms, and a summary string, rendered in the interface as colour-coded tag elements as shown in Figure 7.

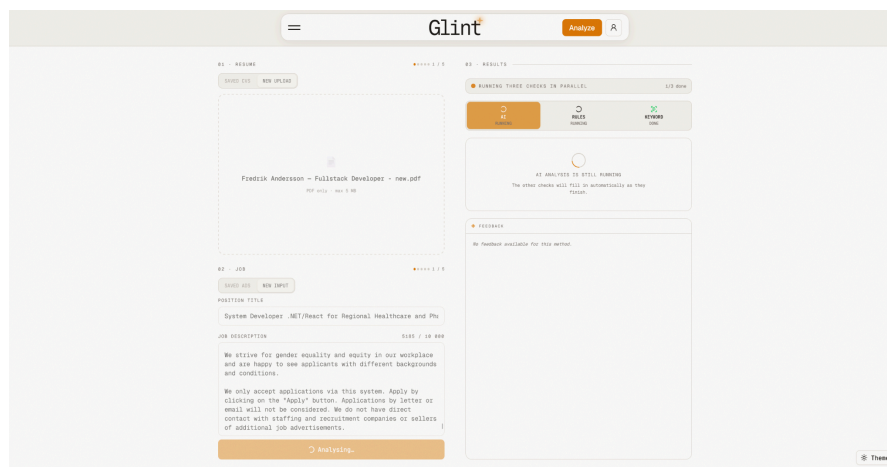


Figure 6: The Glint analysis page mid-analysis, demonstrating the real-time Server-Sent Events (SSE) streaming behaviour with one method completed and two still processing. Screenshot taken by the authors, May 2026.

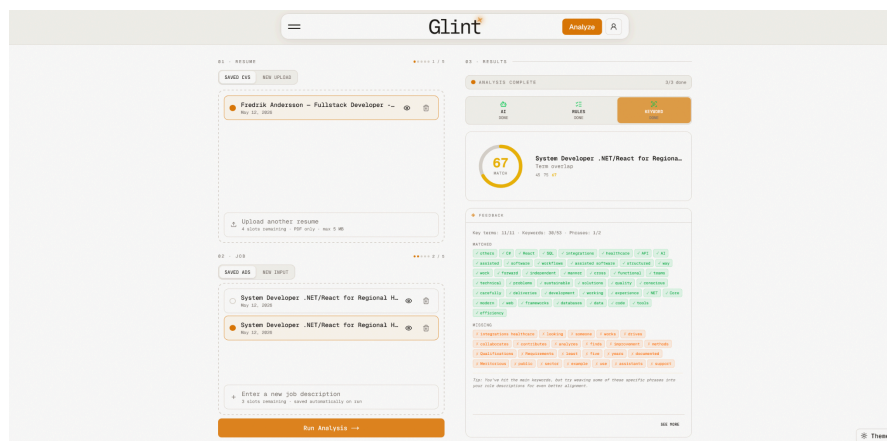


Figure 7: The keyword analysis feedback view displaying colour-coded matched and missing terms extracted from the job advertisement. Screenshot taken by the authors, May 2026.

4. DISCUSSION

The stated purpose of this project was to design and implement a web-based platform that analyses and evaluates, in real time, the degree of alignment between a user-uploaded CV and a provided job advertisement by means of three concurrently executing analysis methods, and to reflect critically on the strengths and limitations of each approach. That purpose was met: Glint delivers real-time scoring from three independent methods, each surfacing a different dimension of CV-to-job fit. The sections below examine how well each method fulfilled its intended role, where the implementation fell short, and what would be required to close those gaps.

4.1 Analysis Methods

The decision to combine three distinct analysis methods was sound in principle, and the design intention is that each method surfaces a different dimension of fit that the others do not address. However, the implementation of each approach revealed challenges that would require considerably more time and resources to fully resolve, whether the combination produces a meaningfully more accurate overall score than any single method in isolation remains untested, as no comparative evaluation against a labelled dataset was conducted within the project timeline

The keyword-frequency analysis proved to be the most difficult method to get right. Technical terminology varies significantly across industries and job types, and the vocabulary list underpinning the method is inherently limited to the terms that could be curated manually. This is compounded by the absence of multi-language support, as the entire pipeline is built around English, meaning the method's accuracy is bounded not only by how comprehensive the vocabulary list is, but also by the assumption that both the CV and the job advertisement conform to a single language. Achieving truly reliable keyword analysis would either require a substantial investment of time in expanding and maintaining the vocabulary, or the identification of a suitable library capable of handling this classification automatically.

The AI-dependent methods introduce a different category of risk: external service availability. Both the semantic analysis and the rule-based evaluation rely on Google Gemini being accessible at the time of the request. If the model's API were to become unavailable or the free-tier quota were exhausted, neither method would function. The most robust long-term solution would be to self-host a language model on the backend, which would eliminate this dependency entirely. Given more time, more extensive prompt engineering would also have been pursued to bring the model's output closer to a genuinely accurate evaluation.

4.2 Known Limitations

Several areas of the application have known limitations that fall outside what was addressed within the project timeline. The score-over-time chart on the user dashboard presents a data integrity issue: when an analysis method fails to produce a result, the chart records a score of zero rather than omitting that data point entirely. This means failed analyses are visually indistinguishable from genuinely low-scoring results, which risks distorting the trend line and misleading the user about their actual progress over time. The correct behaviour would be to exclude failed results from the dataset, though a satisfactory solution was not implemented and remains an open item.

The contact form currently has no working submission endpoint, meaning user feedback cannot be collected through the interface. Access tokens are stored in `localStorage`, which exposes them to cross-site scripting vulnerabilities as identified in the delimitations. Switching to `httpOnly` cookies would address this, but the change was deprioritised given the time it would require. Minor refinements to the visual design are also ongoing, as there are components and pages that are not fully satisfactory from an aesthetic standpoint.

The project also lacks automated unit and integration tests. While the Repository-Service architecture was deliberately structured to support testability, with concrete repositories easily replaceable by mocks, no formal test suite was written within the project timeline. This is a meaningful omission.

The analysis pipeline in particular, with its concurrent execution and multiple external API dependencies, would benefit significantly from unit tests that mock the AI services and integration tests that verify end-to-end scoring behaviour under controlled inputs. Adding automated test coverage is a clear prerequisite for any production hardening of the codebase.

4.3 Architectural Decisions

A significant architectural pivot occurred during the integration of the analysis pipeline that is worth reflecting on. The initial design called for three separate API endpoints, one per analysis method, requiring the frontend to manage three concurrent HTTP requests independently. In practice this introduced unnecessary complexity, as handling three distinct loading states and error boundaries client-side increased the risk of inconsistent UI states and made the code considerably harder to reason about. The architecture was subsequently consolidated into a single endpoint, with the backend taking sole responsibility for orchestrating the three methods and returning a unified response. This produced cleaner code, more predictable error handling, and a more coherent user experience.

That this limitation only became apparent during integration rather than planning reflects a broader lesson: certain architectural trade-offs are difficult to anticipate on paper and only surface under the practical demands of a working system. The willingness to refactor a core design decision mid-project, rather than persisting with an approach that was proving problematic, reduced overall complexity and produced more predictable error handling.

4.4 Deployment

The deployment infrastructure, while functional, reflects the constraints of a zero-budget project. Both Vercel and Railway are appropriate platforms for a project of this scale, but Railway's entry-level tier introduces cold start latency, meaning the first request after a period of inactivity may experience a noticeable delay while the server reinitialises.

A paid hosting tier would eliminate this behaviour. The domain name obtained fulfils the project's requirements and remains valid for twelve months.

4.5 Future Prospects

Looking further ahead, there is a genuine question of whether Glint could evolve into a commercially viable product. Jobscan and Rezi both operate on freemium models, where Jobscan markets itself as helping candidates land three times more interviews (Jobscan, 2025) while offering limited guest functionality behind a paywall. Glint currently provides all three analysis methods to guest users with no restriction beyond a rate limit, which is more generous than either competitor's free tier. Importantly, the inclusion of semantic and rule-based methods alongside keyword matching is intended to reduce the incentive for keyword stuffing that single-method tools have been criticised for producing. If multi-language support were added, the analysis accuracy improved, and the interface polished to a production standard, there would be a reasonable case for introducing a similar model. Offering three complementary methods rather than a single keyword scan could be a meaningful differentiator in that market, particularly given the broader problem identified by Fuller et al. (2021) that existing automated screening tools are already failing both candidates and employers. A tool that combines transparency, semantic understanding, and structural evaluation in a single accessible platform addresses a gap that purely keyword-driven products do not.

5. CONCLUSION

This thesis set out to design and implement a web-based platform capable of analysing and quantifying, in real time, the alignment between a user-uploaded CV and a provided job advertisement. That goal was achieved. The result is Glint, a functioning full-stack application that executes three independent analysis methods concurrently and delivers their results to the user as each completes, providing a more transparent and multi-dimensional evaluation than any single method could offer in isolation.

The project demonstrates that combining semantic AI analysis, rule-based evaluation, and keyword-frequency matching produces a more nuanced picture of CV-to-job fit than the keyword-only approaches employed by existing commercial tools such as Jobscan and Rezi. Each method captures something the others miss: the AI method understands context and meaning, the rule-based method evaluates structured criteria with defined weights, and the keyword method provides fast, deterministic feedback on terminology coverage. Used together, they give the user a richer basis for improving their application.

At the same time, the project reveals the practical limits of each approach. The keyword method is constrained by the completeness of a manually curated vocabulary. The AI methods are dependent on external API availability and the quality of prompt engineering. These limitations do not invalidate the results, but they establish a clear direction for future work: broader vocabulary coverage, support for additional languages, self-hosted language models, and more extensive prompt refinement would each meaningfully improve the accuracy of the analysis.

From a technical perspective, the project reinforced the value of iterative development and the willingness to revisit architectural decisions when empirical evidence points toward a better solution. The consolidation of three separate API endpoints into a single unified pipeline mid-project is one example of this, as is the adoption of PdfPig after a custom parser proved insufficient.

The problem that motivated this project: That qualified candidates are systematically filtered out by automated screening tools that prioritise keyword matching over genuine competence, remains unsolved at scale. Glint does not solve it either, but it represents a step toward giving candidates the tools to understand and respond to that reality. Whether the platform could evolve into a commercially viable product depends on the improvements outlined above, but the foundation is in place.

6. SOURCE LIST

Fuller, J.B., Raman, M., Bailey, A., Vaduganathan, N., Vaithianathan, A., Kaufman, E. and Laverty, M. (2021). Hidden Workers: Untapped Talent. Harvard Business School and Accenture. Available at:

<https://www.hbs.edu/managing-the-future-of-work/Documents/research/hiddenworkers09032021.pdf> (Accessed: 12 May 2026).

Jobscan. (2025). The Free Resume Scanner That Helps You Land 3X More Job Interviews. Available at: <https://www.jobscan.co/resume-scanner> (Accessed: 12 May 2026).

Jones, M., Bradley, J. and Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force. Available at: <https://www.rfc-editor.org/rfc/rfc7519> (Accessed: 12 May 2026).

OWASP Foundation. (2021). Testing for Stored Cross Site Scripting. Open Web Application Security Project. Available at: https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/02-Testing_for_Stored_Cross_Site_Scripting (Accessed: 12 May 2026).

Rezi. (2026). ATS Resume Checker: Free & Instant. Available at: <https://www.rezi.ai/tools/resume-checker> (Accessed: 12 May 2026).

WHATWG. (2026). Server-sent events. HTML Living Standard. Available at: <https://html.spec.whatwg.org/multipage/server-sent-events.html> (Accessed: 13 May 2026).

APPENDICES

Appendix A:

List of Server-Side Dependencies and NuGet Packages. Where the version used in this project is the one on the top of each row, whereas the bottom number is the latest version available.

	BCrypt.Net-Next by Chris McKee, Ryan D. Emerl, Damien Miller A fixed, enhanced and namespace compatible version of BCrypt.Net port of jBCrypt implemented in C#. It uses a variant of the Blowfish encryption algorithm's keying schedule, and introduces a work factor, which allows you to determine how expensive the hash function will be, allowing the algorit...	4.1.0 4.2.0
	Google.GenAI by Google LLC Google GenAI SDK for .NET	1.6.1
	Microsoft.AspNetCore.Authentication.JwtBearer by Microsoft ASP.NET Core middleware that enables an application to receive an OpenID Connect bearer token.	8.0.12 10.0.8
	Microsoft.AspNetCore.Identity.EntityFrameworkCore by Microsoft ASP.NET Core Identity provider that uses Entity Framework Core.	8.0.12 10.0.8
	Microsoft.EntityFrameworkCore by Microsoft Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL Database, SQLite, Azure Cosmos DB, MySQL, PostgreSQL, and other databases through a provider plugin API.	8.0.12 10.0.8
	Microsoft.EntityFrameworkCore.Design by Microsoft Shared design-time components for Entity Framework Core tools.	8.0.12 10.0.8
	Microsoft.EntityFrameworkCore.Tools by Microsoft Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.	8.0.12 10.0.8
	Npgsql by Shay Rojansky, Nikita Kazmin, Brar Piening, Nino Floris, Yoh Deadfall, Austin Drenski, Emil Lenngren, Francisco Figueiredo Jr., Kenji Uno Npgsql is the open source .NET data provider for PostgreSQL.	8.0.9 10.0.2
	Npgsql.EntityFrameworkCore.PostgreSQL by Shay Rojansky, Austin Drenski, Yoh Deadfall PostgreSQL/Npgsql provider for Entity Framework Core.	8.0.11 10.0.1
	PdfPig by UglyToad Reads text content from PDF documents and supports document creation. Apache 2.0 licensed.	0.1.14
	Swashbuckle.AspNetCore by domaindrivendev Swagger tools for documenting APIs built on ASP.NET Core	6.6.2 10.1.7

Appendix B:

Link to our GitHub Organization where the project can be found.

<https://github.com/Glint-io>

Links to GitHub profiles of both authors.

<https://github.com/MyNameJaeff>

<https://github.com/maxberglund2>